

**Laporan Tugas Mini Project Pengganti UTS : Pengembangan Aplikasi
myMemoria Sebagai Aplikasi Manajemen Catatan Harian Berbasis
PySide6**



Dosen Pengampu
Pahrul Irfan, S.Kom., M.Kom.

Disusun oleh:

Nama : Alysa Meliana

NIM : F1D02310035

Kelas : Informatika - D

Teknik Informatika
Fakultas Teknik
Universitas Mataram

2026

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi saat ini telah banyak memengaruhi cara manusia dalam mencatat dan menyimpan informasi. Salah satu bentuk catatan pribadi yang sering digunakan adalah diary atau catatan harian. Namun, pencatatan secara manual pada buku fisik memiliki beberapa keterbatasan, seperti risiko kehilangan, sulit dalam pencarian data, serta tidak adanya fitur pengelolaan yang lebih fleksibel. Oleh karena itu, diperlukan sebuah aplikasi digital yang dapat membantu pengguna dalam mencatat dan mengelola cerita harian secara lebih efisien, terstruktur, dan mudah diakses kapan saja. Aplikasi myMemoria dikembangkan sebagai solusi untuk permasalahan tersebut dengan memanfaatkan teknologi GUI berbasis PySide6 dan database SQLite.

1.2 Tujuan

Adapun tujuan dari pengembangan aplikasi myMemoria adalah sebagai berikut:

1. Mengembangkan aplikasi berbasis PySide6
2. Menerapkan konsep CRUD
3. Menggunakan database SQLite untuk penyimpanan data
4. Menerapkan prinsip Separation of Concerns (SoC)
5. Menerapkan styling menggunakan QSS (Qt Style Sheet)

1.3 Manfaat

Manfaat dari aplikasi myMemoria yaitu:

1. Mempermudah pengguna dalam mencatat cerita harian
2. Mempermudah pencarian dan pengelolaan catatan harian
3. Menyediakan penyimpanan data yang lebih terstruktur

DESKRIPSI APLIKASI

2.1 Deskripsi Aplikasi

myMemoria merupakan aplikasi berbasis *Graphical User Interface* (GUI) yang dibangun menggunakan framework PySide6. Aplikasi ini dirancang untuk membantu pengguna dalam mencatat, menyimpan, serta mengelola catatan harian (diary digital) secara lebih sistematis dan terorganisir. Dengan adanya aplikasi ini, proses pencatatan yang sebelumnya dilakukan secara manual pada buku catatan dapat digantikan dengan sistem digital yang lebih efisien dan mudah diakses kapan saja. Aplikasi ini menyimpan seluruh data catatan ke dalam database SQLite. Setiap catatan memiliki beberapa atribut seperti tanggal, judul, isi catatan, kategori, dan mood yang membantu pengguna dalam mengelompokkan serta mengidentifikasi isi catatan dengan lebih mudah.

2.2 Fungsi Aplikasi

Fungsi utama dari aplikasi myMemoria adalah sebagai media pencatatan digital yang bisa membantu pengguna untuk mengelola berbagai catatan harian dalam satu sistem yang terstruktur. Aplikasi ini tidak hanya berfungsi sebagai tempat menulis catatan, tetapi juga sebagai sistem manajemen data sederhana yang dapat menyimpan, menampilkan, memperbarui, dan menghapus data catatan sesuai kebutuhan pengguna. Selain itu, aplikasi ini juga berfungsi sebagai sarana untuk membantu pengguna dalam mengarsipkan pengalaman atau cerita pribadi secara lebih rapi. Dengan adanya fitur pencarian dan pengelompokan kategori, pengguna dapat dengan mudah menemukan kembali catatan tertentu berdasarkan judul atau isi yang diinginkan.

2.3 Fitur Aplikasi

Aplikasi myMemoria memiliki beberapa fitur yang mendukung pengelolaan data catatan harian, yaitu:

- **Tambah Catatan**

Fitur ini digunakan untuk menambahkan catatan baru ke dalam sistem. Pengguna dapat mengisi form yang terdiri dari tanggal, judul, isi catatan, kategori, dan mood. Setelah data disimpan, informasi tersebut akan langsung tersimpan ke dalam database SQLite dan ditampilkan pada tabel utama aplikasi.

- **Tampilkan Catatan**

Fitur ini digunakan untuk menampilkan seluruh data catatan yang telah tersimpan dalam database. Data ditampilkan dalam bentuk tabel agar lebih mudah dikelola oleh

pengguna, serta setiap baris pada tabel mewakili satu catatan lengkap dengan informasinya.

- **Update Catatan**

Fitur ini dapat membantu pengguna untuk memperbarui atau mengubah data catatan yang sudah ada. Pengguna dapat memilih catatan tertentu, kemudian melakukan perubahan pada isi, kategori, atau mood sesuai kebutuhan.

- **Hapus Catatan**

Fitur ini digunakan untuk menghapus catatan yang tidak diperlukan lagi. Sistem akan menampilkan dialog konfirmasi sebelum data benar-benar dihapus untuk mencegah kesalahan penghapusan data penting.

- **Pencarian Catatan**

Fitur ini digunakan untuk mencari catatan berdasarkan judul, sehingga dengan ini pengguna tidak perlu mencari secara manual di seluruh daftar catatan, karena sistem akan menampilkan hasil yang sesuai dengan kata kunci yang dimasukkan.

- **Import & Export Data**

Fitur ini dapat membantu pengguna untuk mengimpor data dari file teks serta mengekspor catatan ke dalam format TXT dan PDF. Fitur export PDF menggunakan library ReportLab sehingga catatan dapat disimpan dalam bentuk dokumen yang lebih formal dan siap untuk dicetak.

2.4 Teknologi yang Digunakan

Dalam proses pengembangan aplikasi myMemoria, digunakan beberapa teknologi utama untuk membangun sistem, yaitu:

- **PySide6**

PySide6 digunakan sebagai framework utama dalam pembuatan antarmuka pengguna (GUI). Framework ini menyediakan berbagai komponen seperti QMainWindow, QDialog, QWidget, QPushButton, dan lainnya yang digunakan untuk membangun tampilan aplikasi yang responsif.

- **SQLite**

SQLite digunakan sebagai sistem manajemen database lokal yang menyimpan seluruh data catatan pengguna. SQLite dipilih karena bersifat ringan, sehingga tidak memerlukan server tambahan, dan sangat cocok digunakan untuk aplikasi skala kecil yang sederhana.

- **QSS**

QSS digunakan untuk mengatur tampilan visual aplikasi agar lebih menarik. Dengan QSS, setiap komponen seperti tombol, tabel, menu, dan dialog dapat diberi styling khusus sehingga aplikasi memiliki tema yang konsisten yaitu dominan berwarna merah muda pada aplikasi myMemoria..

- **ReportLab**

ReportLab digunakan untuk menghasilkan file PDF pada fitur export. Library ini bisa membuat aplikasi untuk mengubah data catatan menjadi dokumen PDF, sehingga lebih tersusun rapi dan dapat digunakan untuk kebutuhan dokumentasi.

DESAIN DAN IMPLEMENTASI APLIKASI

3.1 Perancangan Sistem

Aplikasi myMemoria dirancang sebagai aplikasi berbasis PySide6 yang digunakan untuk mengelola catatan harian atau cerita pengguna dalam bentuk digital. Dalam proses pengembangannya, aplikasi ini menerapkan konsep *Separation of Concerns* (SoC), yaitu sebuah pendekatan dalam pemrograman yang memisahkan setiap bagian sistem berdasarkan tanggung jawabnya masing-masing. Tujuannya supaya kode program menjadi lebih terstruktur dan lebih mudah untuk dikembangkan maupun diperbaiki di kemudian hari. Dalam arsitektur sistem ini, aplikasi dibagi menjadi beberapa lapisan utama. Lapisan pertama adalah *User Interface* (UI) yang bertugas menangani seluruh tampilan dan interaksi pengguna menggunakan komponen PySide6 seperti QMainWindow, QDialog, QTableWidgetItem, QLineEdit, dan lainnya. Lapisan kedua adalah Controller, yang berfungsi sebagai penghubung antara UI dan database, serta mengatur seluruh logika seperti proses penambahan, pengeditan, penghapusan, dan pencarian data. Lapisan ketiga adalah Database Layer, yang bertugas menyimpan seluruh data menggunakan SQLite. Selain itu terdapat juga Style Layer yang menggunakan file QSS eksternal untuk mengatur tampilan antarmuka agar lebih menarik.

3.2 Struktur Folder

Struktur project myMemoria disusun secara modular agar setiap bagian tidak saling bercampur. File utama aplikasi berada pada main.py yang digunakan untuk menjalankan program. Folder UI berisi seluruh komponen tampilan seperti main_window.py sebagai halaman utama, dialog_form.py sebagai form untuk menambah dan mengedit data, serta search_dialog.py untuk fitur pencarian data. Selanjutnya terdapat folder controllers yang berisi diary_controller.py, yaitu bagian yang mengatur logika utama aplikasi dan menjadi penghubung antara UI dan database. Folder database berisi database.py yang bertugas untuk membuat koneksi ke SQLite serta membuat tabel jika belum tersedia. Terakhir terdapat folder styles yang berisi file main.qss untuk mengatur seluruh tampilan aplikasi menggunakan CSS versi Qt.

3.3 Perancangan Antarmuka

Struktur yang Antarmuka aplikasi myMemoria dirancang agar mudah dan nyaman saat digunakan oleh pengguna. Tampilan utama aplikasi menggunakan QMainWindow yang menampilkan data dalam bentuk tabel menggunakan QTableWidgetItem. Tabel ini digunakan untuk menampilkan seluruh data catatan seperti tanggal, judul, kategori, dan mood. Selain tampilan utama, aplikasi juga memiliki beberapa dialog tambahan seperti

Add/Edit Dialog yang digunakan untuk menambahkan dan mengubah data. Kemudian terdapat Search Dialog yang berfungsi untuk mencari data berdasarkan judul cerita. Pada bagian atas aplikasi juga terdapat Menu Bar yang terdiri dari menu File, Search, dan View. Menu File digunakan untuk operasi data seperti menambah, mengedit, menghapus, serta import dan export data, serta terdapat menu View berisi informasi tentang aplikasi.

3.4 Perancangan Database

Aplikasi myMemoria menggunakan SQLite sebagai sistem database karena sifatnya yang ringan dan tidak memerlukan server tambahan, serta mudah diintegrasikan dengan aplikasi desktop Python. Database ini digunakan untuk menyimpan seluruh data catatan. Di dalam database terdapat satu tabel utama yang memiliki beberapa kolom, yaitu id sebagai primary key, tanggal untuk menyimpan tanggal catatan, judul untuk menyimpan judul cerita, isi_catatan untuk menyimpan isi utama cerita, serta kategori dan mood untuk memberikan keterangan tambahan pada catatan.

3.5 Alur Kerja Sistem

Alur kerja pada aplikasi myMemoria dimulai dari eksekusi file main.py yang akan menjalankan seluruh aplikasi. Setelah itu, sistem akan melakukan koneksi ke database dan mengambil seluruh data catatan melalui controller. Data yang diambil kemudian ditampilkan pada tabel di halaman utama aplikasi. Pengguna dapat melakukan berbagai interaksi seperti menambah catatan baru, mengedit catatan yang sudah ada, menghapus catatan, serta mencari catatan berdasarkan judul. Setiap perubahan yang dilakukan oleh pengguna akan diproses oleh controller terlebih dahulu, kemudian diteruskan ke database untuk disimpan atau diperbarui. Setelah itu, tampilan tabel akan diperbarui secara otomatis agar data yang ditampilkan sesuai dengan data yang ada di database.

3.6 Implementasi Fitur

Aplikasi myMemoria memiliki beberapa fitur yang diimplementasikan menggunakan konsep CRUD. Fitur Create digunakan untuk menambahkan data baru melalui form input pada dialog. Data yang dimasukkan oleh pengguna akan dikirim ke controller untuk disimpan ke dalam database. Fitur Read digunakan untuk menampilkan seluruh data yang ada di database ke dalam tabel pada halaman utama. Fitur Update dapat membantu pengguna untuk mengubah data yang sudah ada, kemudian mengeditnya melalui form dialog. Sementara itu, fitur Delete digunakan untuk menghapus data yang diinginkan oleh pengguna dengan disertai konfirmasi menggunakan QMessageBox. Selain itu terdapat juga fitur Search yang dapat membantu pengguna dalam mencari data yang diinginkan, sehingga lebih mudah untuk di temukan.

3.7 Implementasi QSS

Aplikasi myMemoria menggunakan Qt Style Sheet untuk mengatur tampilan antarmuka. Penggunaan QSS membuat pengembang dapat memisahkan desain tampilan dari logika program, sehingga lebih mudah untuk melakukan perubahan desain tanpa mengubah kode utama. Tema yang digunakan pada aplikasi ini dominan berwarna merah muda yang diterapkan pada berbagai komponen seperti tombol, tabel, menu bar, dialog, dan message box.

3.8 Penerapan Signals and Slots

Dalam pengembangan aplikasi myMemoria, digunakan konsep signals and slots yang merupakan mekanisme utama dalam PySide6 untuk menghubungkan interaksi antar komponen. Setiap aksi pengguna seperti klik tombol, pemilihan menu, atau pengisian form akan memicu signal yang kemudian diproses oleh slot atau fungsi tertentu. Contohnya adalah ketika pengguna menekan tombol “Tambah Story”, maka akan membuka dialog form input. Ketika tombol “Simpan” ditekan, maka data akan dikirim ke controller untuk diproses dan disimpan ke database. Sehingga dengan ini, aplikasi menjadi lebih responsif terhadap tindakan pengguna.

HASIL DAN IMPLEMENTASI

4.1 Hasil Implementasi

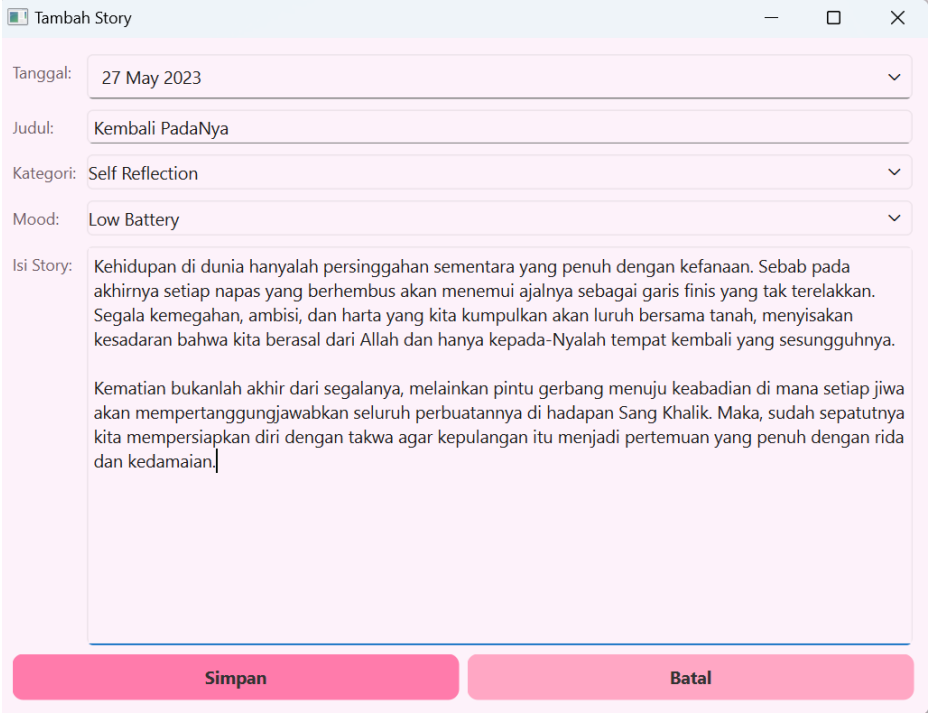
Hasil implementasi dari aplikasi myMemoria menunjukkan bahwa seluruh rancangan sistem yang telah dibuat sebelumnya berhasil direalisasikan menjadi sebuah aplikasi berbasis Python dengan menggunakan framework PySide6 untuk antarmuka grafis dan SQLite sebagai sistem basis data. Aplikasi ini mampu menjalankan fungsi utama berupa pengelolaan catatan harian yang mencakup proses CRUD, di mana pengguna dapat menambah, melihat, mengubah, dan menghapus data catatan dengan mudah. Selain itu, aplikasi juga dilengkapi dengan fitur tambahan seperti pencarian catatan berdasarkan judul, serta fitur import dan export data dalam format TXT dan PDF, sehingga meningkatkan fleksibilitas dalam pengelolaan data. Secara keseluruhan, hasil implementasi menunjukkan bahwa aplikasi myMemoria dapat berjalan dengan baik, serta memiliki antarmuka yang sederhana dan mudah digunakan oleh pengguna.

4.2 Hasil Demo Aplikasi



Berdasarkan gambar di atas, diketahui bahwa gambar tersebut adalah tampilan utama aplikasi myMemoria yang dapat digunakan oleh pengguna untuk mengelola seluruh data catatan harian. Pada bagian ini, sistem menampilkan seluruh data yang tersimpan dalam database dalam bentuk tabel agar pengguna dapat melihat ringkasan informasi secara lebih terstruktur. Tampilan ini dirancang dengan konsep antarmuka sederhana untuk memberikan kenyamanan dan kemudahan dalam penggunaan aplikasi. Pada halaman utama ini terdapat beberapa komponen utama, yaitu tabel data catatan yang berisi informasi seperti tanggal, judul,

kategori, dan mood. Selain itu, terdapat tombol aksi utama seperti tambah, edit, dan hapus yang digunakan untuk melakukan pengelolaan data catatan secara langsung. Pengguna juga dapat mengakses fitur tambahan melalui menu bar yang tersedia, seperti fitur pencarian catatan, import dan export data, serta informasi mengenai aplikasi melalui menu view.



The image shows a 'Tambah Story' dialog box with the following fields and content:

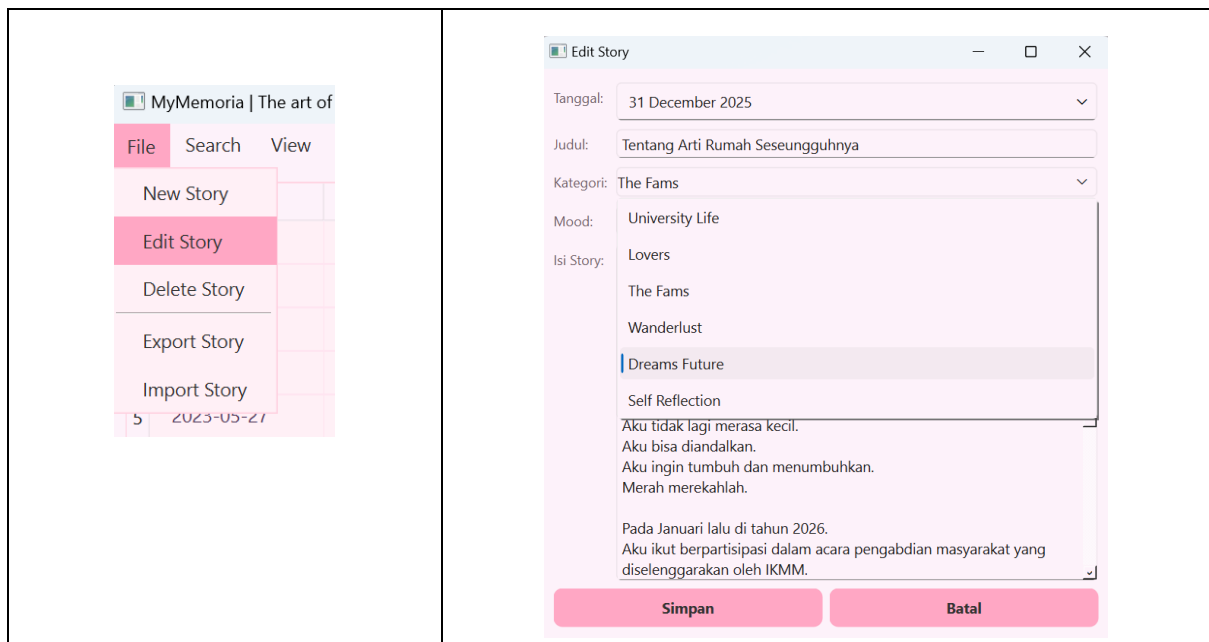
- Tanggal:** 27 May 2023
- Judul:** Kembali PadaNya
- Kategori:** Self Reflection
- Mood:** Low Battery
- Isi Story:**

Kehidupan di dunia hanyalah persinggahan sementara yang penuh dengan kefananaan. Sebab pada akhirnya setiap napas yang berhembus akan menemui ajalnya sebagai garis finis yang tak terelakkan. Segala kemegahan, ambisi, dan harta yang kita kumpulkan akan luruh bersama tanah, menyisakan kesadaran bahwa kita berasal dari Allah dan hanya kepada-Nyalah tempat kembali yang sesungguhnya.

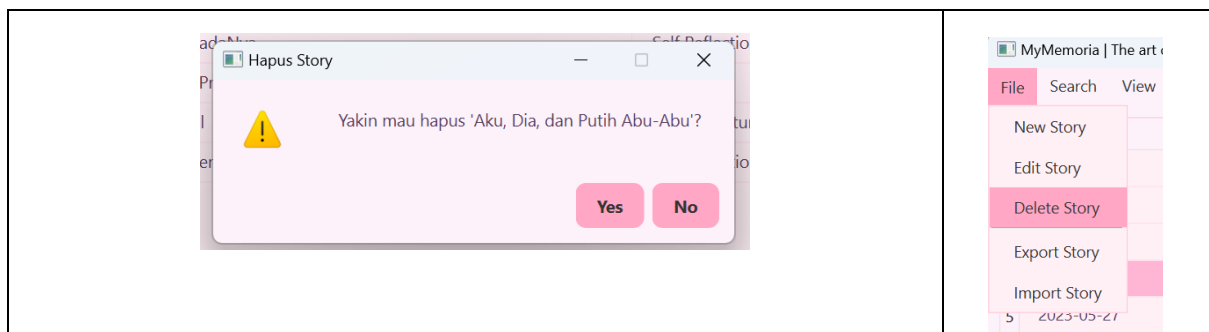
Kematian bukanlah akhir dari segalanya, melainkan pintu gerbang menuju keabadian di mana setiap jiwa akan mempertanggungjawabkan seluruh perbuatannya di hadapan Sang Khalik. Maka, sudah sepatutnya kita mempersiapkan diri dengan takwa agar kepulangan itu menjadi pertemuan yang penuh dengan rida dan kedamaian.

At the bottom of the dialog are two buttons: **Simpan** and **Batal**.

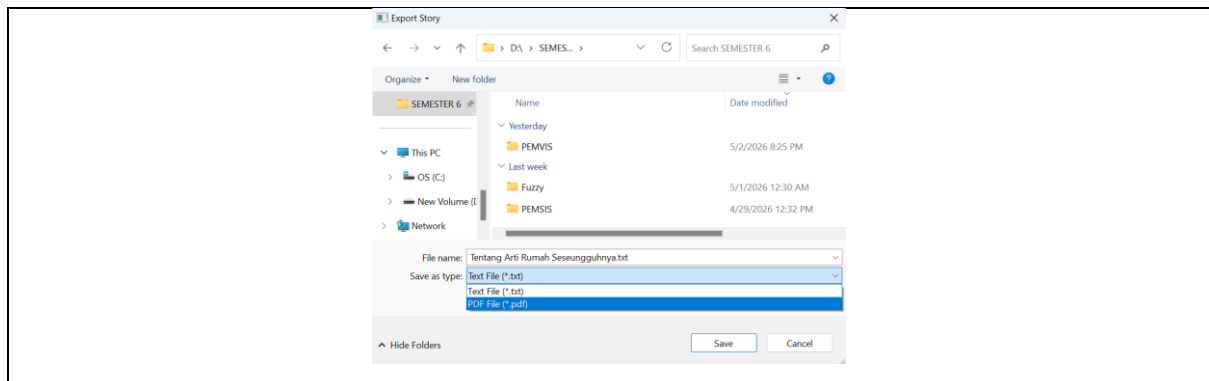
Berdasarkan gambar di atas, dapat diketahui bahwa gambar tersebut adalah fitur Tambah Story pada aplikasi myMemoria digunakan untuk menambahkan catatan atau cerita baru ke dalam sistem. Fitur ini diakses melalui tombol “Tambah Story” pada halaman utama atau dapat juga melalui menu File (New Story). Ketika fitur ini dipilih, sistem akan menampilkan sebuah dialog form yang berisi beberapa *input* yang harus diisi oleh pengguna, yaitu tanggal, judul, kategori, mood, serta isi catatan. Pada bagian tanggal, sistem secara otomatis akan mengisi tanggal hari ini, namun pengguna masih dapat mengubahnya sesuai kebutuhan. Selanjutnya, pengguna diminta untuk mengisi judul catatan, memilih kategori dan mood pada catatan. Bagian isi catatan berupa text area yang digunakan untuk menuliskan cerita dengan bebas. Tombol “Simpan” untuk menyimpan data ke dalam database. Sistem kemudian akan memvalidasi input, dan apabila data sudah lengkap maka catatan akan ditambahkan ke dalam database SQLite serta langsung ditampilkan kembali pada tabel di halaman utama.



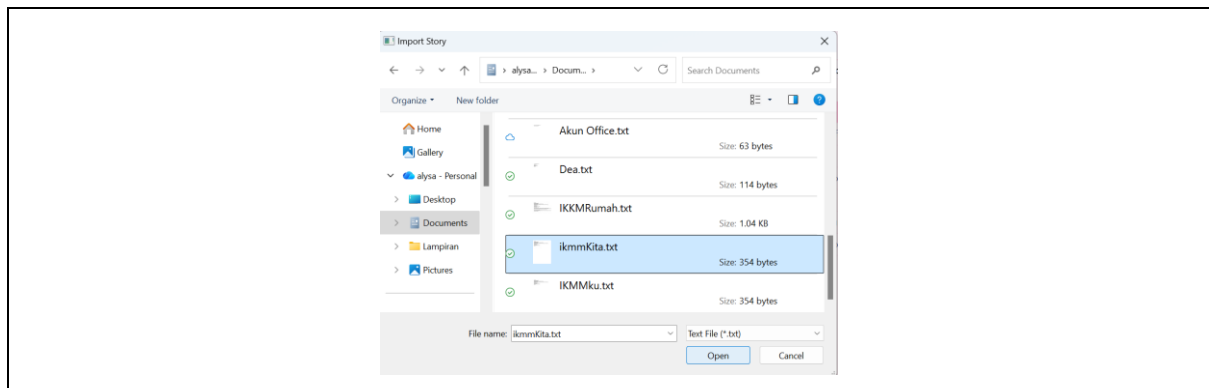
Berdasarkan gambar di atas, dapat diketahui bahwa gambar di atas adalah fitur Edit untuk memberikan fleksibilitas kepada pengguna jika ingin mengubah informasi pada catatan yang sudah tersimpan. Fungsi ini bekerja dengan mengambil *ID* unik dari baris yang dipilih sebagai acuan untuk menjalankan perintah *update* pada database SQLite. Dengan ini, integritas data tetap terjaga karena pengguna dapat memperbarui judul, kategori, mood, dan isi cerita tanpa harus membuat catatan baru dari awal.



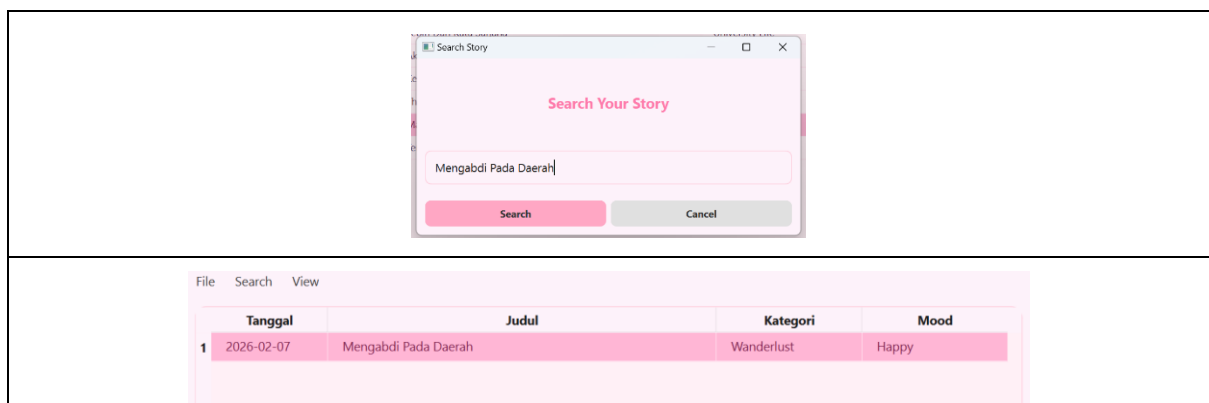
Berdasarkan gambar di atas, dapat diketahui bahwa gambar tersebut merupakan fitur *Delete* atau hapus data. Ketika pengguna memilih sebuah catatan dan menekan tombol hapus, aplikasi tidak langsung menghilangkan data tersebut, melainkan memunculkan sebuah *QMessageBox* sebagai dialog konfirmasi. Apabila pengguna memberikan persetujuan pada dialog tersebut, *controller* akan mengirimkan perintah *delete* ke database SQLite berdasarkan *Primary Key* (*ID*) catatan tersebut. Lalu, setelah proses penghapusan di database berhasil, antarmuka tabel pada halaman utama akan diperbarui.



Berdasarkan gambar di atas, fitur Export dirancang untuk memudahkan pengguna dalam membagikan catatan hariannya dalam format dokumen formal. Aplikasi menggunakan library ReportLab untuk mengonversi data dari SQLite menjadi file PDF dan juga opsi export ke format TXT untuk kebutuhan dokumen teks sederhana.

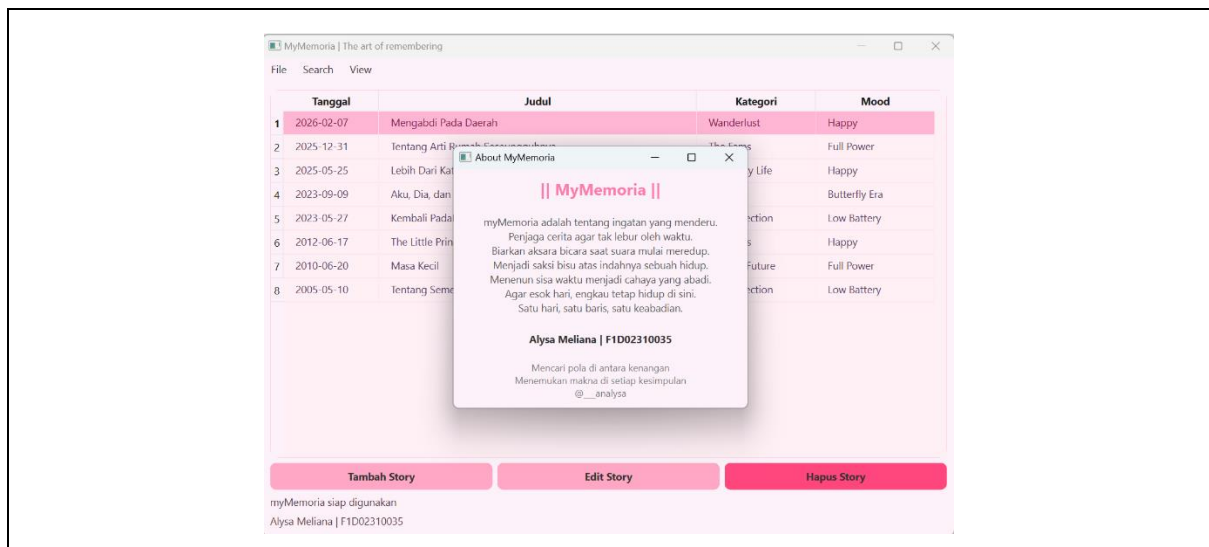


Berdasarkan gambar di atas, fitur Import berfungsi untuk memasukkan data catatan dari luar aplikasi ke dalam database myMemoria. Fitur ini berguna bagi pengguna yang sebelumnya memiliki catatan dalam file teks dan ingin mengarsipkannya ke dalam sistem digital ini. Aplikasi akan membaca baris-baris pada file eksternal tersebut, melakukan *parsing* data, lalu menyimpannya ke dalam database melalui *controller*. Sehingga pengguna tidak perlu memasukkan data secara manual satu per satu jika sudah memiliki draf catatan sebelumnya.



Berdasarkan gambar di atas, fitur *Search* digunakan untuk memudahkan pengguna dalam menemukan catatan tertentu di antara sekumpulan catatan story yang ada. Setiap kali pengguna

memasukkan teks, aplikasi akan menjalankan *query* SELECT dengan bantuan operator LIKE pada database SQLite. Sehingga tabel pada antarmuka utama akan terfilter dan hanya menampilkan baris yang sesuai dengan kata kunci.



Berdasarkan gambar di atas, fitur *About* berfungsi untuk memberikan keterangan atau informasi terkait aplikasi berupa nama dan deskripsi aplikasi, serta informasi terkait pengembang aplikasi myMemoria.

KESIMPULAN

5.1 Kesimpulan

Berdasarkan hasil implementasi aplikasi myMemoria, dapat disimpulkan bahwa aplikasi ini berhasil dibangun sebagai sistem pencatatan catatan harian/diary dengan menggunakan Python dan framework PySide6 serta SQLite sebagai basis data. Aplikasi ini mampu berjalan dengan baik dan menyediakan fitur utama berupa pengelolaan data catatan yang mencakup proses Create, Read, Update, dan Delete, sehingga pengguna dapat dengan mudah menambahkan, melihat, mengubah, dan menghapus catatan sesuai kebutuhan. Selain fitur utama, aplikasi myMemoria juga dilengkapi dengan fitur pendukung seperti pencarian catatan berdasarkan judul, serta fitur import dan export data dalam format TXT dan PDF. Fitur-fitur tersebut diharapkan dapat memberikan kemudahan bagi pengguna dalam mengelola dan memindahkan data catatan hariannya.

LAMPIRAN

1. Link GitHub

<https://github.com/F1D02310035/pv26-miniproject-mymemoria-alya-F1D02310035>

2. Link YouTube

3. Code Program

```
import sys
import os
sys.path.append(os.path.dirname(__file__))
from PySide6.QtWidgets import QApplication
from database.database import create_table
from ui.main_window import MainWindow
if __name__ == "__main__":
    create_table()
    app = QApplication(sys.argv)
    style_path = os.path.join(os.path.dirname(__file__), "styles",
"main.qss")
    if os.path.exists(style_path):
        with open(style_path, "r", encoding="utf-8") as f:
            app.setStyleSheet(f.read())
    window = MainWindow()
    window.show()
    sys.exit(app.exec())
```

main.py

```
from PySide6.QtWidgets import (
    QMainWindow, QTableWidgetItem, QTableWidgetItem,
    QVBoxLayout, QHBoxLayout, QWidget,
    QPushButton, QLabel, QMessageBox, QDialog, QHeaderView,
    QFileDialog, QInputDialog)
import os
from PySide6.QtGui import QAction
from PySide6.QtCore import Qt, QDate
from ui.dialog_form import AddEditDialog
from controllers.diary_controller import DiaryController
from ui.search_dialog import SearchDialog
class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.controller = DiaryController()
        self.setWindowTitle("MyMemoria | The art of remembering")
        self.setMinimumSize(870, 600)
        self.table = QTableWidgetItem()
        self.table.setColumnCount(5)
        self.table.setHorizontalHeaderLabels(
            ["ID", "Tanggal", "Judul", "Kategori", "Mood"])
        self.table.setColumnHidden(0, True)
        self.table.setSelectionBehavior(QTableWidgetItem.SelectRows)
        self.table.setSelectionMode(QTableWidgetItem.SingleSelection)
        header = self.table.horizontalHeader()
        header.setSectionResizeMode(QHeaderView.Fixed)
        self.table.setColumnWidth(1, 120)
        self.table.setColumnWidth(2, 400)
        self.table.setColumnWidth(3, 150)
        self.table.setColumnWidth(4, 150)
        self.btn_tambah = QPushButton("Tambah Story")
```

```

self.btn_edit = QPushButton("Edit Story")
self.btn_hapus = QPushButton("Hapus Story")
self.btn_hapus.setObjectName("btn_hapus")
layout_btn = QHBoxLayout()
layout_btn.addWidget(self.btn_tambah)
layout_btn.addWidget(self.btn_edit)
layout_btn.addWidget(self.btn_hapus)

self.status_label = QLabel("myMemoria siap digunakan")
self.label_nama_nim = QLabel("Alysa Meliana | F1D02310035")

layout = QVBoxLayout()
layout.addWidget(self.table)
layout.addLayout(layout_btn)
layout.addWidget(self.status_label)
layout.addWidget(self.label_nama_nim)

container = QWidget()
container.setLayout(layout)
self.setCentralWidget(container)

menubar = self.menuBar()

# FILE MENU
menu_file = menubar.addMenu("File")

action_new = QAction("New Story", self)
action_new.triggered.connect(self._on_tambah_clicked)

action_edit = QAction("Edit Story", self)
action_edit.triggered.connect(self._on_edit_clicked)

action_delete = QAction("Delete Story", self)
action_delete.triggered.connect(self._on_hapus_clicked)

action_export = QAction("Export Story", self)
action_export.triggered.connect(self._on_export_clicked)

action_import = QAction("Import Story", self)
action_import.triggered.connect(self._on_import_clicked)

menu_file.addAction(action_new)
menu_file.addAction(action_edit)
menu_file.addAction(action_delete)
menu_file.addSeparator()
menu_file.addAction(action_export)
menu_file.addAction(action_import)

# SEARCH MENU
menu_search = menubar.addMenu("Search")

action_search = QAction("Search Story", self)
action_search.triggered.connect(self._on_search_clicked)

menu_search.addAction(action_search)

# VIEW MENU
menu_view = menubar.addMenu("View")

action_about = QAction("About", self)
action_about.triggered.connect(self._show_about)

```



```

        menu_view.addAction(action_about)

        self.btn_tambah.clicked.connect(self._on_tambah_clicked)
        self.btn_edit.clicked.connect(self._on_edit_clicked)
        self.btn_hapus.clicked.connect(self._on_hapus_clicked)

        self.load_entries()

    def load_entries(self):
        entries = self.controller.get_entries()
        self._fill_table(entries)

    def _fill_table(self, entries):
        self.table.setRowCount(len(entries))

        for row_index, entry in enumerate(entries):
            id_, tanggal, judul, isi, kategori, mood = entry

            self.table.setItem(row_index, 0,
                               QTableWidgetItem(str(id_)))
            self.table.setItem(row_index, 1,
                               QTableWidgetItem(tanggal))
            self.table.setItem(row_index, 2, QTableWidgetItem(judul))
            self.table.setItem(row_index, 3,
                               QTableWidgetItem(kategori))
            self.table.setItem(row_index, 4, QTableWidgetItem(mood))

        # TAMBAH
        def _on_tambah_clicked(self):
            dialog = AddEditDialog(self)

            if dialog.exec() == QDialog.Accepted:
                data, error = dialog.get_data()

                if data is None:
                    QMessageBox.warning(self, "Error", error)
                    return

                self.controller.add_entry(data)
                self.load_entries()

        # EDIT
        def _on_edit_clicked(self):

            if self.table.currentRow() < 0:
                QMessageBox.warning(self, "Peringatan", "Pilih catatan
dulu!")
                return

            row = self.table.currentRow()
            entry_id = int(self.table.item(row, 0).text())

            entries = self.controller.get_entries()
            selected = None

            for e in entries:
                if e[0] == entry_id:
                    selected = e
                    break

```

```

        dialog = AddEditDialog(self, selected)

        if dialog.exec() == QDialog.Accepted:
            data, _ = dialog.get_data()
            self.controller.update_entry(entry_id, data)
            self.load_entries()

# DELETE
def _on_hapus_clicked(self):
    row = self.table.currentRow()

    if row < 0:
        QMessageBox.warning(self, "Peringatan", "Pilih catatan
dulu!")
        return

    entry_id = int(self.table.item(row, 0).text())
    judul = self.table.item(row, 2).text()

    msg = QMessageBox(self)
    msg.setWindowTitle("Hapus Story")
    msg.setText(f"Yakin mau hapus '{judul}'?")
    msg.setIcon(QMessageBox.Warning)

    msg.setStandardButtons(QMessageBox.Yes | QMessageBox.No)
    msg.setDefaultButton(QMessageBox.No)

    confirm = msg.exec()

    if confirm == QMessageBox.Yes:
        self.controller.delete_entry(entry_id)
        self.load_entries()

# SEARCH
def _on_search_clicked(self):
    dialog = SearchDialog()

    if dialog.exec():
        keyword = dialog.get_value()

        if keyword:
            results = self.controller.search_entries(keyword)
            self._fill_table(results)
        else:
            self.load_entries()

# EXPORT
def _on_export_clicked(self):
    row = self.table.currentRow()

    if row < 0:
        QMessageBox.warning(self, "Export", "Pilih story dulu!")
        return

    entry_id = int(self.table.item(row, 0).text())
    tanggal = self.table.item(row, 1).text()
    judul = self.table.item(row, 2).text()
    kategori = self.table.item(row, 3).text()
    mood = self.table.item(row, 4).text()

    entries = self.controller.get_entries()

```

```

isi = ""

for e in entries:
    if e[0] == entry_id:
        isi = e[3]
        break

file_path, _ = QFileDialog.getSaveFileName(
    self,
    "Export Story",
    f"{judul}",
    "Text File (*.txt);;PDF File (*.pdf)"
)

if not file_path:
    return

try:
    if file_path.endswith(".txt"):
        with open(file_path, "w", encoding="utf-8") as f:
            f.write("=== myMemoria ===\n\n")
            f.write(f"Judul: {judul}\n")
            f.write(f"Tanggal: {tanggal}\n")
            f.write(f"Kategori: {kategori}\n")
            f.write(f"Mood: {mood}\n")
            f.write(f"Isi: {isi}\n")

        QMessageBox.information(self, "Export", "Berhasil
export TXT!")

    elif file_path.endswith(".pdf"):
        from reportlab.platypus import SimpleDocTemplate,
Paragraph, Spacer
        from reportlab.lib.styles import getSampleStyleSheet

        doc = SimpleDocTemplate(file_path)
        styles = getSampleStyleSheet()

        content = []

        content.append(Paragraph("=== myMemoria ===",
styles["Title"]))
        content.append(Spacer(1, 12))

        content.append(Paragraph(f"<b>Judul:</b> {judul}",
styles["Normal"]))
        content.append(Paragraph(f"<b>Tanggal:</b>
{tanggal}", styles["Normal"]))
        content.append(Paragraph(f"<b>Kategori:</b>
{kategori}", styles["Normal"]))
        content.append(Paragraph(f"<b>Mood:</b> {mood}",
styles["Normal"]))
        content.append(Spacer(1, 5))
        content.append(Paragraph(f"<b>Isi:</b><br/>{isi}",
styles["Normal"]))

        doc.build(content)

        QMessageBox.information(self, "Export", "Berhasil
export PDF!")

```

```

except Exception as e:
    QMessageBox.critical(self, "Error", str(e))

# IMPORT
def _on_import_clicked(self):
    file_path, _ = QFileDialog.getOpenFileName(
        self,
        "Import Story",
        "",
        "Text File (*.txt)"
    )

    if not file_path:
        return

    try:
        # FILE
        judul = os.path.splitext(os.path.basename(file_path))[0]

        with open(file_path, "r", encoding="utf-8") as f:
            isi = f.read().strip()

        # INPUT USER
        kategori, ok1 = QInputDialog.getItem(
            self,
            "Kategori Story",
            "Pilih kategori:",
            [
                "University Life",
                "Lovers",
                "The Fams",
                "Wanderlust",
                "Dreams Future",
                "Self Reflection"
            ],
            0,
            False
        )

        if not ok1:
            return

        mood, ok2 = QInputDialog.getItem(
            self,
            "Mood Story",
            "Pilih mood:",
            [
                "Happy",
                "Low Battery",
                "Unbothered",
                "Chill",
                "Butterfly Era",
                "Cloudy",
                "Full Power"
            ],
            0,
            False
        )

        if not ok2:
            return

```

```

        tanggal = QDate.currentDate().toString("yyyy-MM-dd")

        # SAVE
        self.controller.add_entry({
            "tanggal": tanggal,
            "judul": judul,
            "isi_catatan": isi,
            "kategori": kategori,
            "mood": mood
        })

        self.load_entries()

        QMessageBox.information(self, "Import", "Berhasil import
+ setting kategori & mood!")

    except Exception as e:
        QMessageBox.critical(self, "Error", str(e))

# ABOUT
def _show_about(self):
    dialog = QDialog(self)
    dialog.setWindowTitle("About MyMemoria")
    dialog.setFixedSize(370, 300)

    layout = QVBoxLayout()

    # TITLE
    title = QLabel("|| MyMemoria ||")
    title.setObjectName("about_title")
    title.setAlignment(Qt.AlignCenter)

    desc = QLabel(
        "myMemoria adalah tentang ingatan yang menderu.\n"
        "Penjaga cerita agar tak lebur oleh waktu.\n"
        "Biarkan aksara bicara saat suara mulai meredup.\n"
        "Menjadi saksi bisu atas indahnya sebuah hidup.\n"
        "Menenun sisa waktu menjadi cahaya yang abadi.\n"
        "Agar esok hari, engkau tetap hidup di sini.\n"
        "Satu hari, satu baris, satu keabadian."
    )
    desc.setObjectName("about_desc")
    desc.setAlignment(Qt.AlignCenter)
    desc.setWordWrap(True)

    identity = QLabel("Alysa Meliana | F1D02310035")
    identity.setObjectName("about_identity")
    identity.setAlignment(Qt.AlignCenter)

    author = QLabel(
        "Mencari pola di antara kenangan\n"
        "Menemukan makna di setiap kesimpulan\n"
        "@__analysa"
    )
    author.setObjectName("about_author")
    author.setAlignment(Qt.AlignCenter)

    layout.addWidget(title)
    layout.addSpacing(8)
    layout.addWidget(desc)

```

```
layout.addSpacing(10)
layout.addWidget(identity)
layout.addStretch()
layout.addWidget(author)
dialog.setLayout(layout)
dialog.exec()
```

Main_window.py

```
from PySide6.QtCore import Qt, QDate
from PySide6.QtWidgets import (
    QDialog, QFormLayout, QLineEdit, QTextEdit,
    QComboBox, QVBoxLayout, QPushButton, QHBoxLayout,
    QDateEdit, QMessageBox
)
from PySide6.QtGui import QFont

class AddEditDialog(QDialog):
    def __init__(self, parent=None, entry_data=None):
        super().__init__(parent)

        self.setWindowTitle(
            "Tambah Story" if entry_data is None else "Edit Story"
        )

        self.setMinimumSize(500, 500)
        self.resize(500, 500)

        self.entry_id = None

        font = QFont("Segoe UI", 9)
        self.setFont(font)

        self.date_edit = QDateEdit()
        self.date_edit.setCalendarPopup(True)
        self.date_edit.setDisplayFormat("dd MMMM yyyy")
        self.date_edit.setDate(QDate.currentDate())
        self.date_edit.setButtonSymbols(QDateEdit.NoButtons)

        self.line_judul = QLineEdit()
        self.line_judul.setPlaceholderText("Masukkan judul story...")

        self.combo_kategori = QComboBox()
        self.combo_kategori.addItem("University Life",
            "Lovers",
            "The Fams",
            "Wanderlust",
            "Dreams Future",
            "Self Reflection"
        ])

        self.combo_mood = QComboBox()
        self.combo_mood.addItem("Happy",
            "Low Battery",
            "Unbothered",
            "Chill",
            "Butterfly Era",
            "Cloudy",
            "Full Power"
        ])
    ])
```

```

self.text_isi = QTextEdit()
self.text_isi.setPlaceholderText("Tulis story hari ini...")

form_layout = QFormLayout()
form_layout.setLabelAlignment(Qt.AlignLeft)

form_layout.addRow("Tanggal:", self.date_edit)
form_layout.addRow("Judul:", self.line_judul)
form_layout.addRow("Kategori:", self.combo_kategori)
form_layout.addRow("Mood:", self.combo_mood)
form_layout.addRow("Isi Story:", self.text_isi)

# BUTTON
self.btn_simpan = QPushButton("Simpan")
self.btn_batal = QPushButton("Batal")

btn_layout = QHBoxLayout()
btn_layout.addWidget(self.btn_simpan)
btn_layout.addWidget(self.btn_batal)

# MAIN LAYOUT
main_layout = QVBoxLayout()
main_layout.addLayout(form_layout)
main_layout.addLayout(btn_layout)

self.setLayout(main_layout)

# EDIT
if entry_data is not None:
    self.entry_id = entry_data[0]

    self.date_edit.setDate(
        QDate.fromString(entry_data[1], "yyyy-MM-dd")
    )

    self.line_judul.setText(entry_data[2])
    self.text_isi.setPlainText(entry_data[3])

    idx_kat = self.combo_kategori.findText(
        entry_data[4], Qt.MatchContains
    )
    if idx_kat >= 0:
        self.combo_kategori.setCurrentIndex(idx_kat)

    idx_mood = self.combo_mood.findText(
        entry_data[5], Qt.MatchContains
    )
    if idx_mood >= 0:
        self.combo_mood.setCurrentIndex(idx_mood)

# CONNECT
self.btn_simpan.clicked.connect(self.validate_and_accept)
self.btn_batal.clicked.connect(self.reject)

def validate_and_accept(self):
    if not self.line_judul.text().strip():
        QMessageBox.warning(self, "Error", "Judul tidak boleh
kosong!")
    return

```

```

        self.accept()

# GET DATA
def get_data(self):
    data = {
        "tanggal": self.date_edit.date().toString("yyyy-MM-dd"),
        "judul": self.line_judul.text().strip(),
        "isi_catatan": self.text_isi.toPlainText().strip(),
        "kategori": self.combo_kategori.currentText(),
        "mood": self.combo_mood.currentText()
    }

    return data, None

```

Dialog_form.py

```

from PySide6.QtWidgets import (
    QDialog, QVBoxLayout, QLineEdit,
    QPushButton, QLabel, QHBoxLayout
)
from PySide6.QtCore import Qt
from PySide6.QtGui import QFont

class SearchDialog(QDialog):
    def __init__(self, parent=None):
        super().__init__(parent)

        self.setWindowTitle("Search Story")
        self.setMinimumSize(450, 220)
        self.resize(500, 230)

        self.setFont(QFont("Segoe UI", 10))

# TITLE
self.title_label = QLabel("Search Your Story")
self.title_label.setAlignment(Qt.AlignCenter)
self.title_label.setStyleSheet("""
    font-size: 14pt;
    font-weight: bold;
    color: #ff7bab;
""")

# INPUT
self.input = QLineEdit()
self.input.setPlaceholderText("Masukkan judul story...")
self.input.setStyleSheet("""
    padding: 10px;
    font-size: 11pt;
    border: 1px solid #ffd1e0;
    border-radius: 8px;
""")

self.input.setFocus()

self.btn_search = QPushButton("Search")
self.btn_cancel = QPushButton("Cancel")

self.btn_search.setStyleSheet("""
    background-color: #ffa7c4;
    padding: 8px;
    border-radius: 8px;
    font-weight: bold;

```



```

        """)

        self.btn_cancel.setStyleSheet("""
            background-color: #e0e0e0;
            padding: 8px;
            border-radius: 8px;
        """)

        # BUTTON LAYOUT
        btn_layout = QHBoxLayout()
        btn_layout.addWidget(self.btn_search)
        btn_layout.addWidget(self.btn_cancel)

        # MAIN LAYOUT
        layout = QVBoxLayout()
        layout.addWidget(self.title_label)
        layout.addSpacing(10)
        layout.addWidget(self.input)
        layout.addSpacing(15)
        layout.addLayout(btn_layout)

        self.setLayout(layout)

        # CONNECT
        self.btn_search.clicked.connect(self.accept)
        self.btn_cancel.clicked.connect(self.reject)

    def get_value(self):
        return self.input.text().strip()

```

Search_dialog.py

```

QMainWindow, QWidget {
    background-color: #fdf2fa;
    font-family: "Segoe UI";
    font-size: 10pt;
}

/* LABEL */
QLabel {
    color: #5a3d4a;
    font-family: "Segoe UI";
}

QMainWindow QLabel {
    font-size: 10pt;
}

QDialog QLabel {
    font-size: 9pt;
    color: #6b5b63;
}

/* TABLE */
QTableWidget {
    background-color: #fff0f6;
    border: 1px solid #ffd1e0;
    border-radius: 12px;
    gridline-color: #ffe5f1;
    outline: none;
}

```

```

QTableWidget::item {
    padding: 8px 12px;
    color: #553d5e;
    font-size: 10pt;
    background-color: transparent;
}

QTableWidget::item:hover {
    background-color: #ffe3ee;
}

QTableWidget::item:selected {
    background-color: #ffb6d5;
    color: #7a1f4b;
}

QTableWidget QAbstractItemView {
    outline: none;
}

/* BUTTON */
QPushButton {
    background-color: #ffa7c4;
    border: none;
    border-radius: 8px;
    padding: 8px 16px;
    color: #333;
    font-weight: bold;
}

QPushButton:hover {
    background-color: #ff7bab;
}

QPushButton#btn_hapus {
    background-color: #ff477e;
}

QPushButton#btn_hapus:hover {
    background-color: #ff1744;
}

/* MENU BAR */
QMenuBar {
    background-color: #fdf2fa;
}

QMenuBar::item {
    padding: 6px 10px;
    color: #333;
}

QMenuBar::item:selected {
    background-color: #ffa7c4;
}

/* MENU DROPDOWN */
QMenu {
    background-color: #fff0f6;
    border: 1px solid #ffd1e0;
}

```

```
QMenu::item {
    padding: 8px 20px;
    color: #333;
}

QMenu::item:selected {
    background-color: #ffa7c4;
}

/* MESSAGE BOX */
QMessageBox {
    background-color: #fdf2fa;
    font-family: "Segoe UI";
}

QMessageBox QLabel {
    font-size: 10pt;
    color: #553d5e;
    padding: 10px;
}

QMessageBox QPushButton {
    background-color: #ffa7c4;
    border-radius: 8px;
    padding: 8px 16px;
}

/* CALENDAR */
QCalendarWidget QWidget#qt_calendar_navigationbar {
    background-color: #fdf2fa;
}

QCalendarWidget QToolButton {
    color: #ff1493;
    font-weight: bold;
}

QCalendarWidget QAbstractItemView {
    selection-background-color: #ff1493;
    selection-color: white;
}

/* DATE EDIT */
QDateEdit {
    font-family: "Segoe UI";
    font-size: 10pt;
    padding: 4px;
}

/* ABOUT DIALOG */
#about_title {
    font-size: 20px;
    font-weight: bold;
    color: #ff7bab;
}

#about_desc {
    font-size: 10pt;
    color: #555;
    padding: 5px;
}
```

```

}

#about_identity {
    font-size: 10pt;
    font-weight: bold;
    color: #333;
}

#about_author {
    font-size: 9pt;
    color: #777;
}

```

main.qss

```

import sqlite3

DB_NAME = "myMemoria.db"

def create_connection():
    conn = sqlite3.connect(DB_NAME)
    return conn

def create_table():
    with create_connection() as conn:
        cursor = conn.cursor()
        cursor.execute("""
        CREATE TABLE IF NOT EXISTS entries (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            tanggal TEXT NOT NULL,
            judul TEXT NOT NULL,
            isi_catatan TEXT NOT NULL,
            kategori TEXT NOT NULL,
            mood TEXT NOT NULL
        )
        """)

def insert_entry(tanggal, judul, isi, kategori, mood):
    with create_connection() as conn:
        cursor = conn.cursor()
        cursor.execute("""
            INSERT INTO entries (tanggal, judul, isi_catatan,
            kategori, mood)
            VALUES (?, ?, ?, ?, ?)
            """, (tanggal, judul, isi, kategori, mood))

def get_all_entries():
    with create_connection() as conn:
        cursor = conn.cursor()
        cursor.execute("""
            SELECT * FROM entries
            ORDER BY tanggal DESC
            """)
        return cursor.fetchall()

def search_by_title(keyword):
    with create_connection() as conn:
        cursor = conn.cursor()
        cursor.execute("""
            SELECT * FROM entries
            WHERE judul LIKE ?
            ORDER BY tanggal DESC

```

```

        """, ('%' + keyword + '%',))
        return cursor.fetchall()

def delete_entry_by_id(entry_id):
    with create_connection() as conn:
        cursor = conn.cursor()
        cursor.execute("""
            DELETE FROM entries WHERE id=?
        """, (entry_id,))

def update_entry(entry_id, tanggal, judul, isi, kategori, mood):
    with create_connection() as conn:
        cursor = conn.cursor()
        cursor.execute("""
            UPDATE entries
            SET tanggal=?, judul=?, isi_catatan=?, kategori=?, mood=?
            WHERE id=?
        """, (tanggal, judul, isi, kategori, mood, entry_id))

```

database.py

```

from database.database import (
    insert_entry,
    get_all_entries,
    delete_entry_by_id,
    update_entry as db_update_entry,
    search_by_title
)

class DiaryController:

    def get_entries(self):
        return get_all_entries()

    def add_entry(self, data):
        insert_entry(
            data["tanggal"],
            data["judul"],
            data["isi_catatan"],
            data["kategori"],
            data["mood"]
        )

    def delete_entry(self, entry_id):
        delete_entry_by_id(entry_id)

    def update_entry(self, entry_id, data):
        db_update_entry(
            entry_id,
            data["tanggal"],
            data["judul"],
            data["isi_catatan"],
            data["kategori"],
            data["mood"]
        )

    def search_entries(self, keyword):
        return search_by_title(keyword)

```

Diary_controller.py